**Name - Rutuja Bhalekar**
**Roll No. - 3**

# Experiment No 4

## Aim

To develop a blockchain-powered web application using Solidity on Remix IDE, connect it with MetaMask wallet, and deploy a smart contract on the Sepolia Testnet to record and retrieve student marks.

## Theory

## Blockchain Technology

A blockchain is a distributed, immutable ledger maintained across a network of nodes rather than a single central server. Each block contains a set of transactions, a timestamp, and a cryptographic hash of the previous block, forming a tamper-evident chain. Once data is written to a block and confirmed by the network's consensus mechanism, it cannot be altered or deleted, which is what makes blockchains trustworthy for recording transactions without relying on a central authority.

## Smart Contracts

A smart contract is self-executing program code deployed directly onto a blockchain. Once deployed, its logic runs exactly as written, and its execution cannot be changed, censored, or reversed by any single party — not even the contract's creator, unless the code explicitly allows it (e.g., via an owner-only function). Smart contracts have their own address on the blockchain, can store data permanently in “state variables,” and expose functions that anyone can call to read or modify that data. Every function call that changes data is itself a transaction, requiring computational cost (“gas”) and a cryptographic signature from the caller's wallet.

## Solidity

Solidity is a statically-typed, contract-oriented programming language designed specifically for writing smart contracts on Ethereum-compatible blockchains. It supports familiar programming constructs — structs, mappings, modifiers, events, and access control — while enforcing rules unique to blockchain environments, such as explicit visibility for functions (public, private) and built-in overflow/underflow protection (from Solidity 0.8 onward). Contracts written in Solidity are compiled into EVM (Ethereum Virtual Machine) bytecode before deployment.

## Remix IDE

Remix is a free, browser-based Integrated Development Environment for writing, compiling, debugging, testing, and deploying Solidity smart contracts, with no local installation required. It provides a file explorer, an in-browser Solidity compiler, and a “Deploy & Run Transactions” module that can connect to different environments — a fully simulated in-browser blockchain (Remix VM), or a real network via an injected wallet provider like MetaMask.

## MetaMask Wallet

MetaMask is a cryptocurrency wallet available as a browser extension that manages a user's private keys locally and never exposes them to the websites it interacts with. It acts as an intermediary between a dApp (like Remix) and the blockchain: whenever a transaction needs to be sent — whether deploying a contract or calling a function that modifies data — MetaMask intercepts the request, displays its details (network, gas fee, destination address), and requires explicit user confirmation before signing and broadcasting it. This “injected provider” model is what lets browser-based tools like Remix send real transactions without ever handling the user's private key directly.

## Testnets and Gas

A testnet (such as Sepolia) is a separate blockchain network that mirrors the behavior, rules, and tooling of the Ethereum mainnet but uses worthless test tokens instead of real cryptocurrency. Developers obtain free test ETH from “faucets” to pay for gas — the computational fee required to execute transactions and store data on-chain — allowing contracts to be built, deployed, and tested realistically without financial risk. Once a contract is confirmed to work correctly on a testnet, the same code can be deployed to the mainnet with real funds.

## Output -

The StudentMarks contract was compiled in Remix IDE and deployed on the Sepolia Testnet through the MetaMask wallet (account “Rutuja Bhalekar”). The stored marks were recorded and retrieved on-chain, and the deployment was verified on Blockscout (Sepolia), where the contract source was also auto-verified through Remix. Verified deployment details:

```
Wallet (MetaMask account “Rutuja Bhalekar”):
  0xd682b97bd00f4de6ba5d19b4181377b3acf5f74c
Network: Sepolia Testnet (Chain ID 11155111)
Deployed contract address:
  0x9c5Ed5e4dc1a367F5457caEa536B58A28048BB51
Deployment transaction (Block 12018698):
  0x5bd38c402e0463bdb66e0289004a3b67a8fea815b5eccf4c5d3ee416d303fd6d
addStudent transaction (Block 12018706):
  0x4b315e16b996cdfc163fb93de9ceec827fa5d0caf42acd328adcc08ff2e4b80a
Record stored : addStudent(101, “Rutuja Bhalekar”, 92)
Retrieved     : getStudent(101) = (101, “Rutuja Bhalekar”, 92);  AUTHOR() = “Rutuja Bhalekar”
```

**Fig 1:**

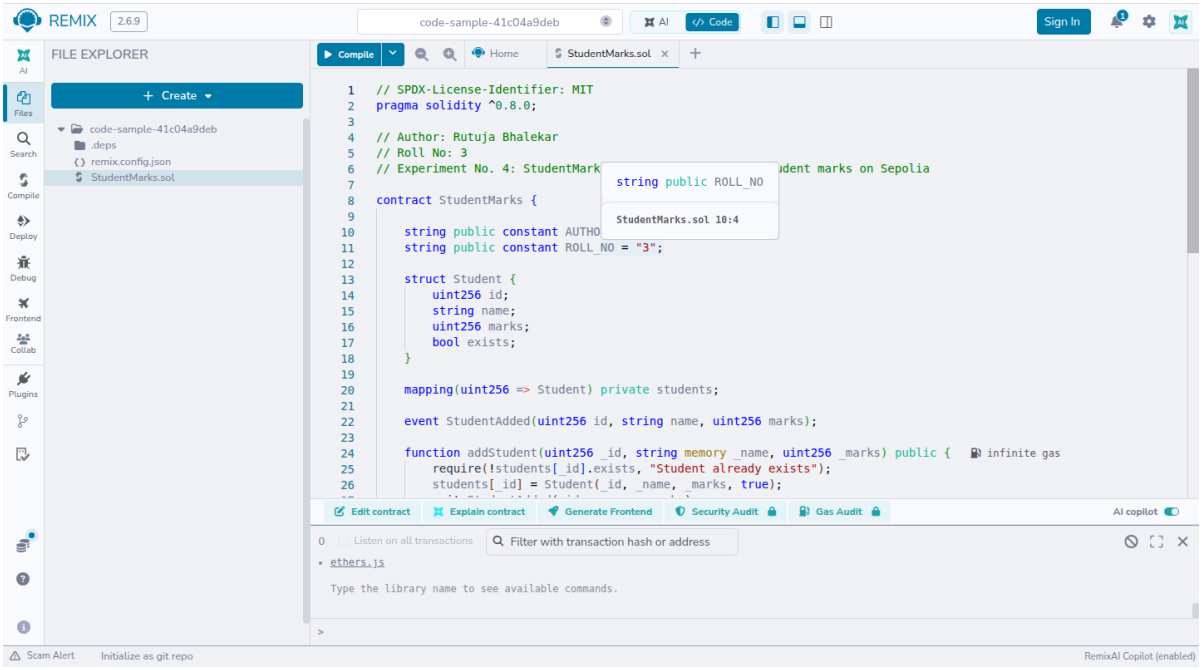



*Fig 1: StudentMarks.sol in Remix IDE showing the author details (Rutuja Bhalekar, Roll No. 3) and the AUTHOR / ROLL_NO constants*

**Fig 2:**

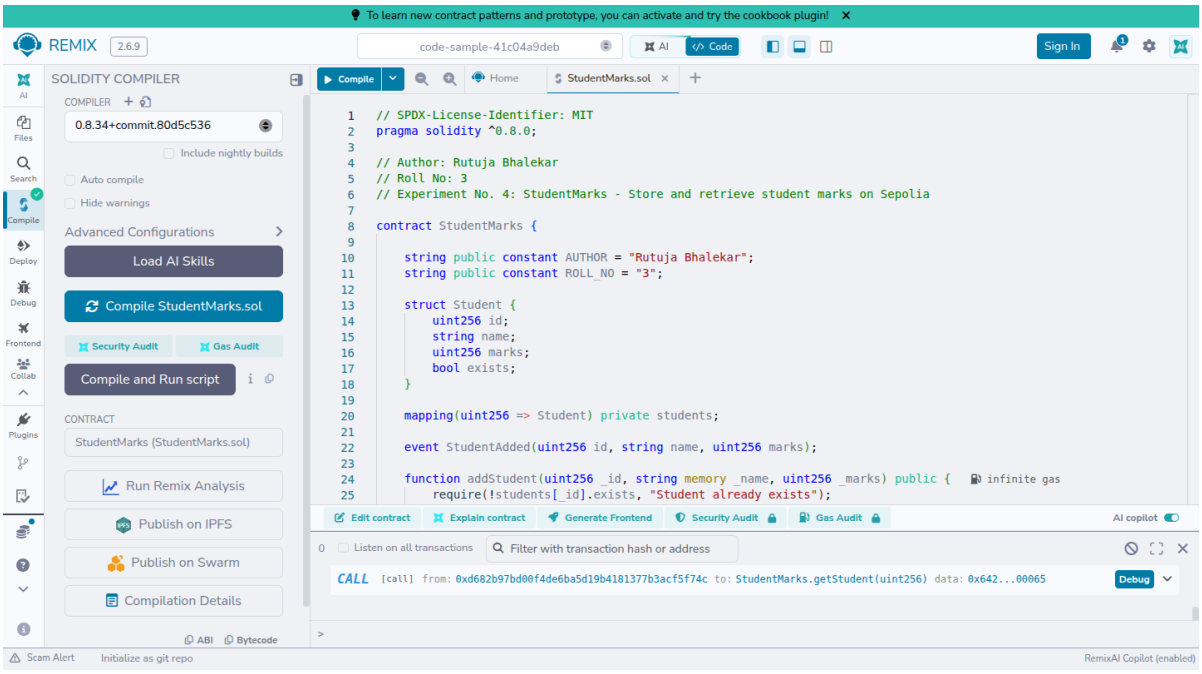



*Fig 2: Successful compilation of the StudentMarks contract in Remix IDE*

**Fig 3:**

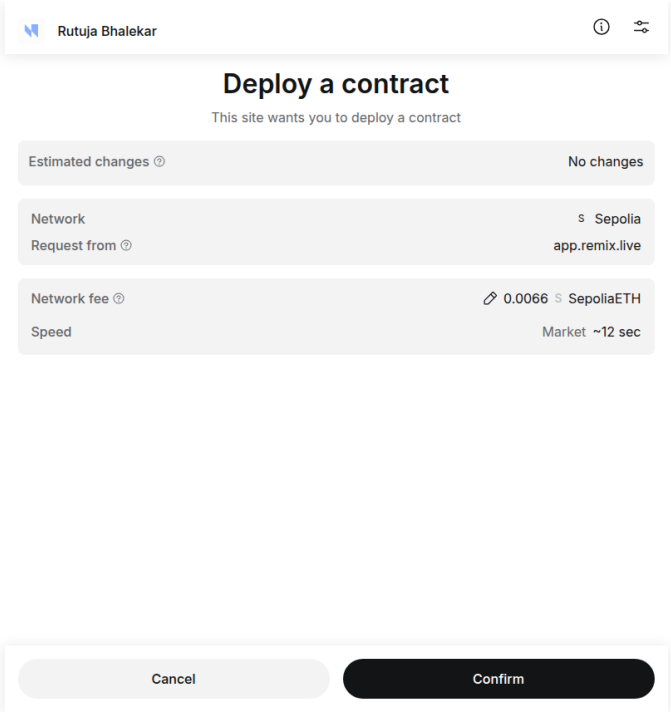



*Fig 3: Confirming the smart contract deployment transaction via MetaMask (Sepolia)*

**Fig 4:**

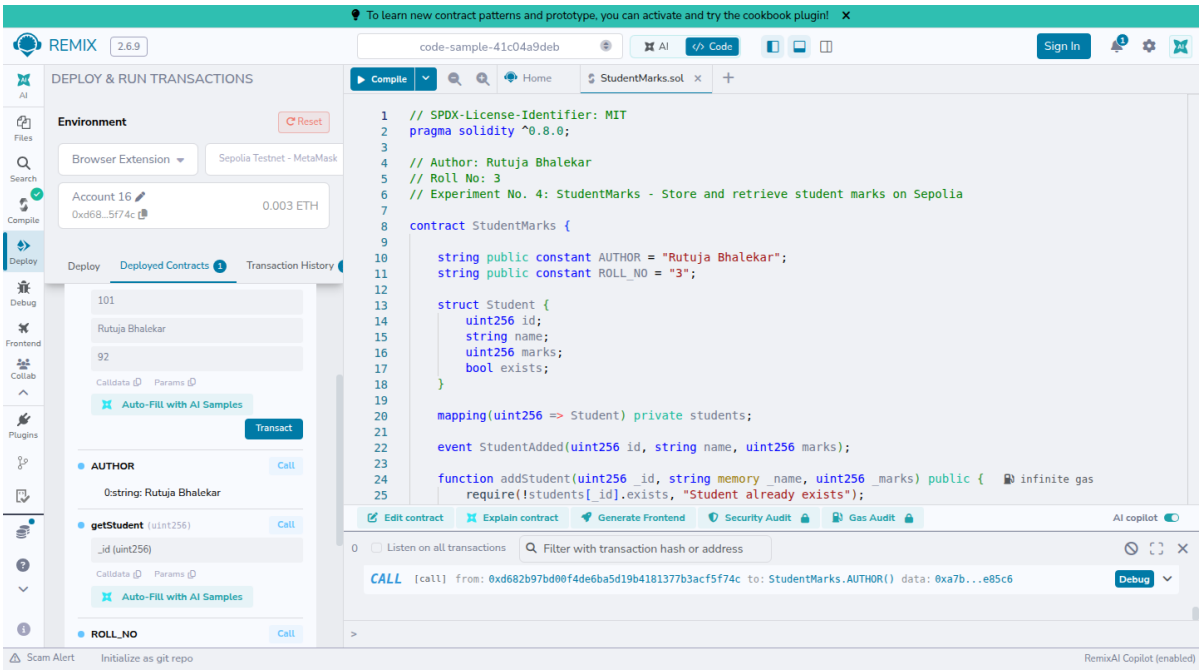



*Fig 4: Deployed contract in Remix — AUTHOR() returns "Rutuja Bhalekar"*

**Fig 5:**

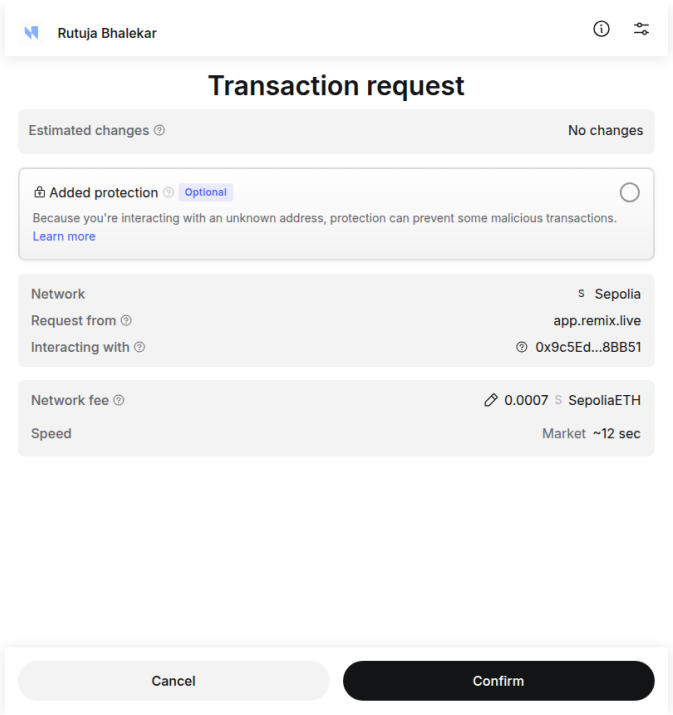



*Fig 5: Confirming the addStudent() transaction via MetaMask*

**Fig 6:**

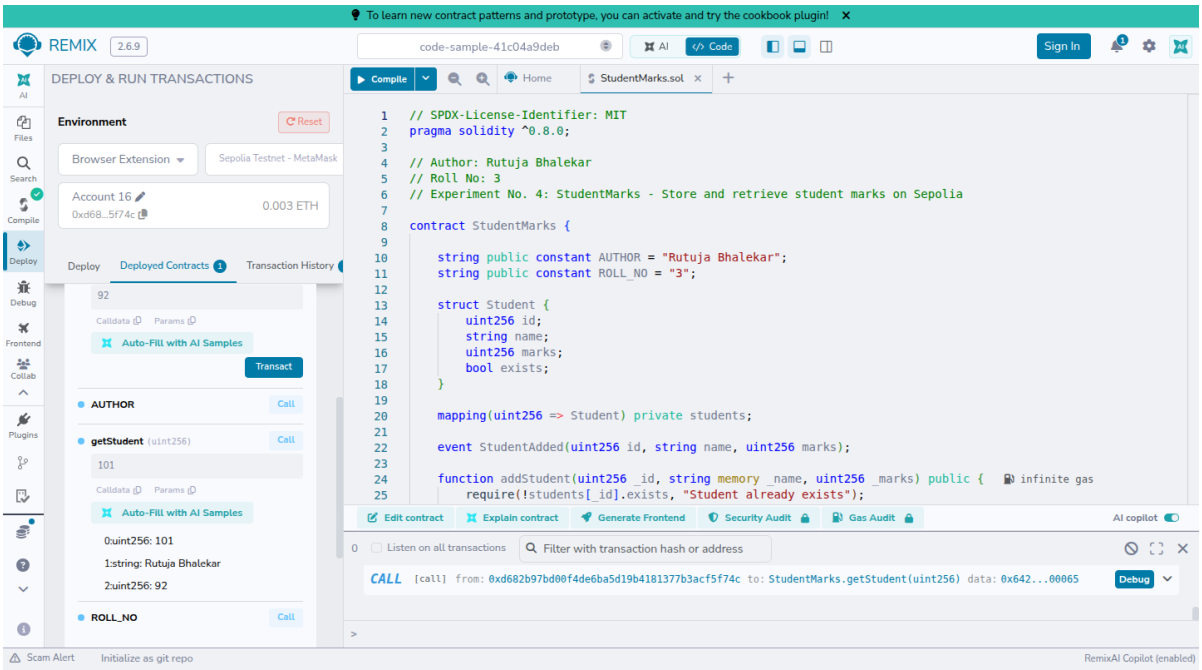



*Fig 6: Retrieving the stored student data using getStudent() — (101, Rutuja Bhalekar, 92)*

**Fig 7:**

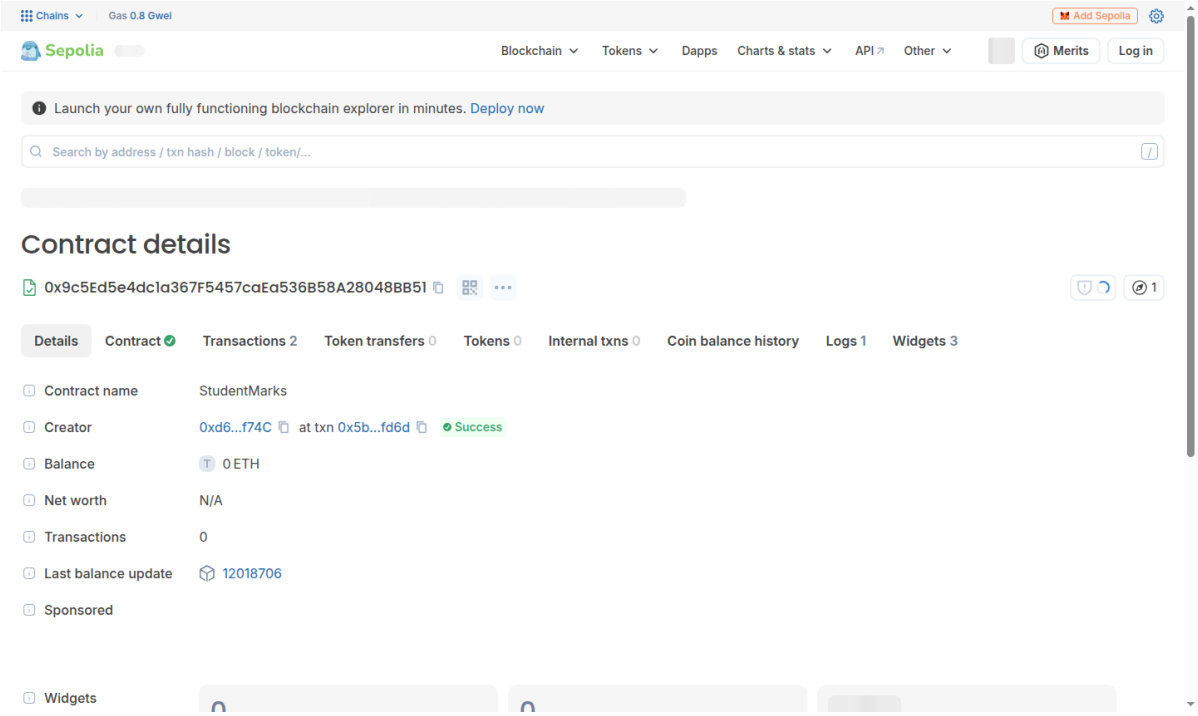



*Fig 7: Verifying the deployed contract and its transactions on Blockscout (Sepolia)*

## Conclusion -

The StudentMarks smart contract was successfully compiled in Remix IDE, deployed on the Sepolia Testnet through MetaMask, and verified on Blockscout. The addStudent() function recorded the marks of Rutuja Bhalekar (Student ID 101, Marks 92) on-chain, and getStudent() retrieved the stored data correctly, with both transactions and the StudentAdded event visible on the explorer. This experiment demonstrated the complete workflow of wallet-based transaction signing and on-chain data storage and retrieval on a real test network.